

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №6
по курсу «Разработка кроссплатформенных приложений»
на тему «Распределенное вычисление определенного интеграла в клиент-серверной системе»

Выполнили студенты группы

23ВВИ1:

Видяев А.А.

Жиганов Н.Д.

Приняли:

к.т.н. доцент Юрова О.В.

к.т.н. доцент Карамышева Н.С.

Пенза 2026

Цель работы

Изучить основы построения клиент-серверных приложений в Java и модифицировать программу из лабораторной работы №5 так, чтобы вычисление определенного интеграла выполнялось распределенно: сервер принимал данные и собирал результаты, а клиенты выполняли вычисления и возвращали частичные суммы.

Задание на лабораторную работу

Модифицировать приложение из лабораторной работы №5. Реализовать клиент-серверную архитектуру распределенных вычислений. Сервер не должен выполнять вычисления самостоятельно, а обязан только принимать входные данные, распределять подзадачи между клиентами и собирать результаты. Клиенты должны вычислять полученные подинтервалы и отправлять частичные результаты обратно. Для варианта 6, как для четного варианта, взаимодействие должно быть реализовано по протоколу TCP.

Ход работы

1. Анализ архитектуры распределенной системы. За основу была взята программа из ЛР5, в которой интеграл уже вычислялся многопоточно. Для ЛР6 приложение было разделено на две части: серверную и клиентскую. Серверная часть представлена графическим приложением `lb1UI`, а клиентская часть реализована в отдельном классе `IntegralClient`.

```

public class IntegralClient {

    public static void main(String[] args) {
        String host = "localhost";
        int port = 6000;

        try (
            Socket socket = new Socket(host, port);
            ObjectOutputStream out = new ObjectOutputStream(socket.getOutputStream());
            ObjectInputStream in = new ObjectInputStream(socket.getInputStream())
        ) {
            System.out.println("Клиент подключен к серверу.");

            while (true) {
                Object obj = in.readObject();

                if (!(obj instanceof ComputationRequest request)) {
                    break;
                }

                RecIntegral rec = new RecIntegral(
                    request.getLower(),
                    request.getUpper(),
                    request.getStep()
                );

                double partial = rec.integrateSqrtMultiThreaded(
                    request.getLower(),
                    request.getUpper(),
                    request.getStep()
                );

                ComputationResponse response = new ComputationResponse(
                    partial,
                    socket.getInetAddress().getHostAddress()
                );

                out.writeObject(response);
                out.flush();
            }
        }
    }
}

```

2. Выбор способа сетевого взаимодействия. Так как вариант 6 является четным, для обмена данными был выбран протокол TCP. Для его реализации использовались классы `ServerSocket` и `Socket`. TCP был выбран благодаря надежной передаче данных и удобству организации обмена объектами между сервером и клиентами.

```

public class ServerWorker {
    private final Socket socket;
    private final ObjectOutputStream out;
    private final ObjectInputStream in;

    public ServerWorker(Socket socket) throws Exception {
        this.socket = socket;
        this.out = new ObjectOutputStream(socket.getOutputStream());
        this.in = new ObjectInputStream(socket.getInputStream());
    }

    public void sendTask(ComputationRequest request) throws Exception {
        out.writeObject(request);
        out.flush();
    }

    public ComputationResponse readResponse() throws Exception {
        Object obj = in.readObject();
        return (ComputationResponse) obj;
    }

    public String getClientInfo() {
        return socket.getInetAddress().getHostAddress() + ":" + socket.getPort();
    }

    public boolean isAlive() {
        return socket != null && socket.isConnected() && !socket.isClosed();
    }

    public void close() {
        try { in.close(); } catch (Exception ignored) {}
        try { out.close(); } catch (Exception ignored) {}
        try { socket.close(); } catch (Exception ignored) {}
    }
}

```

3. Создание классов обмена данными. Для передачи параметров вычисления от сервера к клиенту был создан сериализуемый класс `ComputationRequest`, содержащий нижнюю границу, верхнюю границу и шаг интегрирования. Для отправки частичного результата обратно на сервер был создан сериализуемый класс `ComputationResponse`, содержащий вычисленное значение и строковую информацию о клиенте.

```

public class ComputationRequest implements Serializable {
    private static final long serialVersionUID = 1L;

    private final double lower;
    private final double upper;
    private final double step;

    public ComputationRequest(double lower, double upper, double step) {
        this.lower = lower;
        this.upper = upper;
        this.step = step;
    }

    public double getLower() {
        return lower;
    }

    public double getUpper() {
        return upper;
    }

    public double getStep() {
        return step;
    }
}

public class ComputationResponse implements Serializable {
    private static final long serialVersionUID = 1L;

    private final double partialResult;
    private final String clientName;

    public ComputationResponse(double partialResult, String clientName) {
        this.partialResult = partialResult;
        this.clientName = clientName;
    }

    public double getPartialResult() {
        return partialResult;
    }

    public String getClientName() {
        return clientName;
    }
}

```

4. Реализация клиентской части. Класс `IntegralClient` подключается к серверу по адресу `127.0.0.1` и заданному порту. После подключения клиент ожидает объект `ComputationRequest`. Получив задание, клиент создает объект `RecIntegral` и выполняет вычисление своего подинтервала. Внутри клиента сохранена многопоточная схема из ЛР5: каждый клиент дополнительно делит свой подинтервал на 6 частей и вычисляет его в 6 потоках через `Runnable`.

```

import java.io.Serializable;

public class RecIntegral implements Serializable {
    private static final long serialVersionUID = 1L;

    private static final double MIN_VALUE = 0.000001;
    private static final double MAX_VALUE = 1000000.0;

    public static final int THREAD_COUNT = 6;

    private double lower;
    private double upper;
    private double step;
    private Double result;

    private void validateBound(double value) throws InvalidRangeException {
        if (value < MIN_VALUE || value > MAX_VALUE) {
            throw new InvalidRangeException(
                "Значение должно быть в диапазоне от " + MIN_VALUE + " до " + MAX_VALUE
            );
        }
    }

    private void validateAll(double lower, double upper, double step) throws InvalidRangeException {
        validateBound(lower);
        validateBound(upper);

        if (lower > upper) {
            throw new InvalidRangeException("Нижняя граница не может быть больше верхней.");
        }

        if (step < MIN_VALUE || step > MAX_VALUE) {
            throw new InvalidRangeException(
                "Шаг должен быть в диапазоне от " + MIN_VALUE + " до " + MAX_VALUE
            );
        }
    }
}

```

5. Реализация серверной части. В классе `Ib1UI` был добавлен запуск TCP-сервера через метод `startServer(...)`. Сервер создает объект `ServerSocket`, ожидает подключения клиентов и сохраняет активные соединения в списке `clients`. Для удобства работы с каждым подключением был создан вспомогательный класс `ServerWorker`, содержащий сокет, потоки ввода-вывода объектов и методы `sendTask(...)` и `readResponse()`.

```

private void startServer(int port) {
    System.out.println("Вход в startServer");

    if (serverRunning) {
        System.out.println("Сервер уже запущен");
        JOptionPane.showMessageDialog(this, "Сервер уже запущен.");
        return;
    }

    Thread serverThread = new Thread(() -> {
        try {
            System.out.println("Поток сервера стартовал");
            serverSocket = new java.net.ServerSocket(port);
            serverRunning = true;

            System.out.println("Сервер запущен на порту " + port);

            while (serverRunning) {
                System.out.println("Ожидание подключения клиента...");
                java.net.Socket clientSocket = serverSocket.accept();
                System.out.println("Клиент подключился: " + clientSocket);

                ServerWorker worker = new ServerWorker(clientSocket);

                synchronized (clients) {
                    clients.add(worker);
                    System.out.println("Клиент добавлен. Всего: " + clients.size());
                }
            }
        } catch (Exception e) {
            System.out.println("Ошибка в сервере:");
            e.printStackTrace();
        }
    });
}

```

6. Организация распределенного вычисления. При нажатии кнопки «Вычислить» сервер определяет число подключенных клиентов, делит общий интервал интегрирования на соответствующее количество частей и отправляет каждому клиенту свой объект `ComputationRequest`. После этого сервер последовательно получает от каждого клиента объект `ComputationResponse` и суммирует частичные результаты. Итоговое значение записывается в выбранную строку таблицы.

```

public class ComputationRequest implements Serializable {
    private static final long serialVersionUID = 1L;

    private final double lower;
    private final double upper;
    private final double step;

    public ComputationRequest(double lower, double upper, double step) {
        this.lower = lower;
        this.upper = upper;
        this.step = step;
    }

    public double getLower() {
        return lower;
    }

    public double getUpper() {
        return upper;
    }

    public double getStep() {
        return step;
    }
}

```

7. Обеспечение корректной работы интерфейса. Как и в ЛР5, длительные операции были вынесены из потока обработки событий Swing. Отправка задач клиентам, ожидание ответов и суммирование результатов выполняются в фоновом потоке, а обновление JTable и вывод сообщений пользователю происходят через `SwingUtilities.invokeLater(...)`.

```

private static final java.util.logging.Logger logger = java.util.logging.Logger.getLogger(lb1UI.class.getName());
private static LinkedList<RecIntegral> records;
private java.net.ServerSocket serverSocket;
private java.util.List<ServerWorker> clients = new java.util.ArrayList<>();
private boolean serverRunning = false;
/**
 * Creates new form lb1UI
 */
public lb1UI() {
    initComponents();
    System.out.println("Конструктор lb1UI вызван");
    records = new LinkedList<>();
    startServer(6000);
    System.out.println("startServer вызван");
}

```

Вывод

В ходе лабораторной работы были изучены основы построения распределенных вычислительных систем по клиент-серверной архитектуре. Программа из ЛР5 была успешно модифицирована: серверная часть принимает входные данные, распределяет подзадачи и собирает частичные результаты, а клиентская часть выполняет вычисления и возвращает ответ по протоколу TCP.

Дополнительно в клиенте сохранена многопоточная схема вычислений из предыдущей лабораторной работы. В результате была получена работоспособная распределенная система вычисления определенного интеграла.